Министерство образования Республики Беларусь
Учреждение образования
Белорусский Государственный университет информатики и радиоэлектроники

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет

по лабораторной работе № 6

на тему «Организация межсервисного взаимодействия с использованием gRPC и Kafka»

Выполнила: Абу Халава И.М.
студент группы 377901

Проверил: Сильванович Ю.В.

Минск 2026

Введение

Лабораторная работа № 6 посвящена реальному межсервисному обмену по gRPC и Kafka в программном средстве анализа рентабельности продукции. Система объединяет производственный и финансовый учёт, рассчитывает себестоимость и прибыль, предоставляет GraphQL API и отчётность. Исправления бизнес-логики и инфраструктуры собраны в итоговой исполняемой версии lab7_profitability_system; прежние каталоги лабораторных сохраняют историю этапов разработки.

Сервисы владеют отдельными базами данных. Gateway проверяет пользователя и права, сервисный слой выполняет бизнес-операции, репозитории сохраняют данные только своего сервиса. В итоговой версии синхронный обмен выполняется по gRPC, а асинхронные реакции — через Kafka.

1 Архитектура распределённой системы

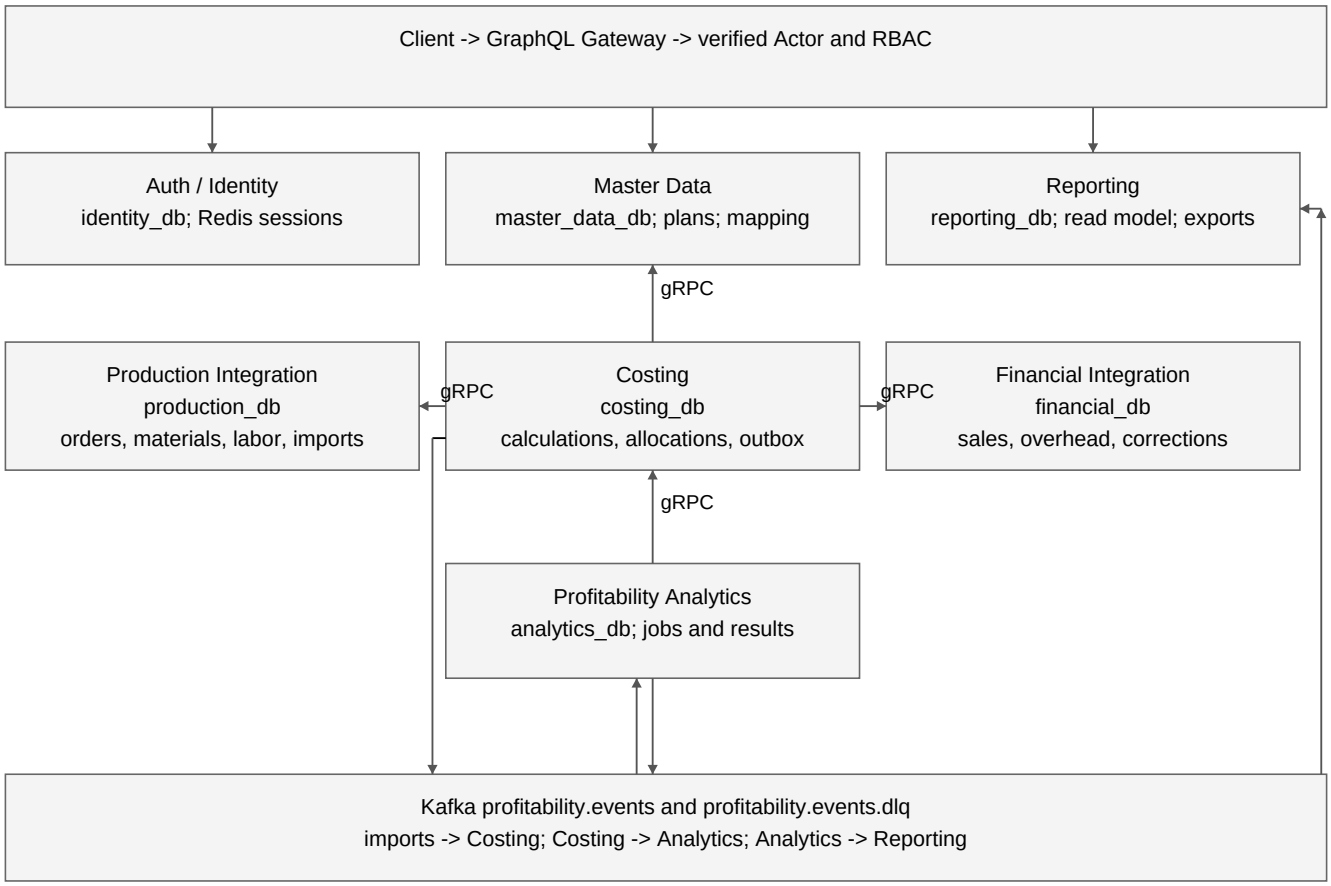


Рисунок. Сервисы, отдельные базы данных и асинхронное обновление Reporting

Gateway обращается к Master Data, Production, Financial, Costing, Analytics и Reporting по сетевым gRPC channel. Каждый сервис работает со своей базой. Kafka передаёт завершившиеся факты и запускает зависимые действия. Auth Service и Redis проверяют пользовательские сессии, а Redis дополнительно кеширует нормативный контекст.

2 gRPC и защита прямого доступа

Типизированный profitability.proto содержит GetCostingContext, LookupExternalMapping, GetProductionCostData, GetOverheadData, GetSalesData, GetApprovedCostCalculation и GetProfitabilityDetail. Ответ Production включает allocation_lines: заказ, центр затрат и значения базы распределения. Это позволяет сохранять проверяемую детализацию OVERHEAD_ALLOCATION, а не только общую сумму.

JSON bridge-контракты служат для прикладных операций Gateway. Все RPC проходят серверный InternalAuthorization; отсутствие действительного служебного JWT даёт UNAUTHENTICATED, неподходящий subject/role — PERMISSION_DENIED. Изменяющие операции разрешены только служебной роли gateway. Пользовательский JWT и бизнес-роли проверяются до этого в Gateway. Порты сервисов доступны внутри сети Compose и не опубликованы наружу.

```
authorization: Bearer <signed internal service JWT>
iss = profitability-internal
aud = internal-rpc
role = INTERNAL_SERVICE
sub = gateway | costing | analytics | production | financial | reporting
exp = short-lived expiry
```

3 Импорт и соответствие событий сохранённым данным

Production и Financial выполняют реальное чтение JSON-снимка, сопоставление внешних кодов и сохранение записей. Производственный импорт создаёт заказы, факты материалов и труда; финансовый — реализацию, накладные расходы и корректировки. Ошибки сохраняются в IMPORT_ERROR, итоговые счётчики отражают реально обработанные записи. Пустой или отсутствующий источник не приводит к ложному ProductionDataImported/FinancialDataImported.

Итог IMPORT_RUN и событие фиксируются через локальный outbox. При COMPLETED публикуется соответствующий DataImported, при FAILED — ImportFailed с реальным importRunId. Dispatcher повторяет отправку при недоступном брокере. Повтор импорта обновляет факты по внешнему ключу и повторно обрабатывает исправленные данные.

4 Реакции Kafka consumers

Событие	Costing	Analytics	Reporting
ProductionDataImported	Готовность Production; инвалидация за период	Исключение устаревшей аналитики	Исключение старой витрины
FinancialDataImported	Готовность Financial; инвалидация за период	Исключение устаревшей аналитики	Исключение старой витрины
ImportFailed	Сброс готовности контура; инвалидация	Инвалидация результатов/заданий	Исключение старой витрины
MasterDataChanged	Инвалидация калькуляций и кеша	Инвалидация результатов/заданий	Исключение зависимых строк
CostCalculationCompleted	Producer	Расчёт указанной версии	Ожидание обновлённой проекции
ProfitabilityCalculated	—	Producer	Upsert read model

Консервативная реакция на MasterDataChanged инвалидирует все калькуляции, поскольку изменение сопоставления либо нормативов может затронуть несколько продуктов. Сохранённые калькуляции не удаляются: их статус становится INVALIDATED и прежние версии остаются доступными для аудита. Для использования необходимо выполнить новый расчёт.

5 Версия калькуляции в Analytics

CostCalculationCompleted содержит calculationId, productId, period, version, суммы и sourceVersions. Consumer передаёт version в GetApprovedCostCalculation. Он не заменяет её последней версией: сообщение о версии 1 обрабатывается с версией 1 даже при уже сохранённой версии 2. Для INVALIDATED сервис возвращает FAILED_PRECONDITION.

Analytics получает реализацию из Financial, рассчитывает стоимость продаж и прибыль, сохраняет PROFITABILITY_RESULT и задание COMPLETED. ProfitabilityCalculated наследует correlationId исходного CostCalculationCompleted. Идентификатор результата повторяемо используется Reporting для обновления проекции.

6 Подключение Reporting read model к GraphQL

profitability и profitabilitySummary в ApplicationFacade направлены в GrpcReportingClient. ReportingGateway.ListProfitability и ProfitabilitySummary читают только profitability_read_model в reporting.db. GraphQL больше не обходит витрину прямым чтением списка из Analytics. Analytics сохраняет ответственность за расчёт и детализацию собственного результата.

Projection обновляется по ProfitabilityCalculated, поэтому выдача имеет eventual consistency: после Mutation новая строка появляется после обработки события. При отсутствии строки возвращается пустая выборка, без скрытого обращения к Analytics. Фильтры группы, продукции, центра затрат и периода одинаковы для списка и сводки; пустая группа не расширяет результат до всех продуктов.

7 Подтверждение сообщений и повреждённые данные

KafkaConsumer получает сырые bytes. Десериализация выполняется внутри обработки записи, чтобы некорректный JSON либо envelope не завершал цикл consumer. После ограниченных повторов обработчика сообщение передаётся в profitability.events.dlq. Повреждённый исходный payload сохраняется в DLQ как base64 вместе с topic, partition и offset.

Offset подтверждается явно для конкретной partition и позиции offset+1 только после успешного handler либо подтверждённой публикации DLQ. Если обе операции не удались, consumer остаётся на текущей записи и повторяет обработку; последующие записи этой partition не подтверждаются поверх неё. Ошибка публикации DLQ не маскируется как успех.

8 Последовательность расчёта

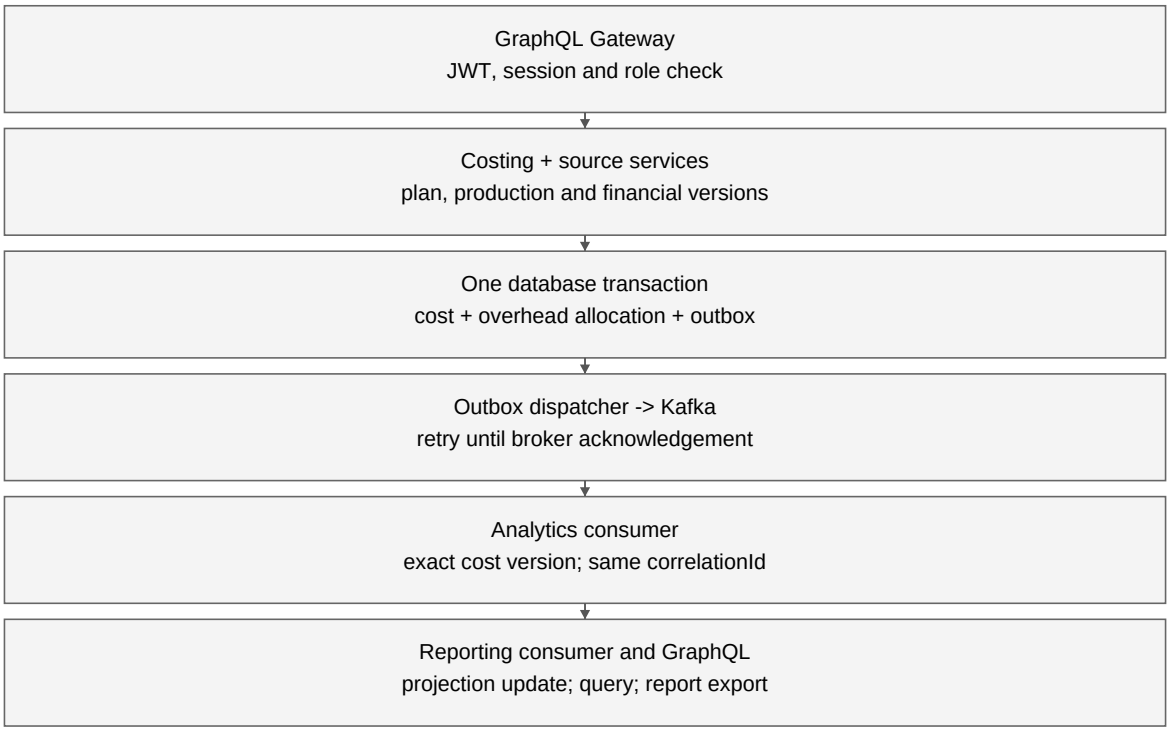


Рисунок. Последовательность расчёта и получения результатов

9 Проверка и запуск

Тесты поднимают реальные gRPC server/client и проверяют типизированные сообщения, защищённость прямого RPC, сохранение фактов импорта, точную версию события, общий correlationId и чтение Reporting. Отдельный тест повреждённой записи подтверждает, что следующая запись обрабатывается только после успешного DLQ и корректного commit. Runtime Kafka и Redis проверяется в отдельном Compose-проекте.

```
LAB7_EVENT_BACKEND=memory LAB7_SESSION_BACKEND=memory \
LAB7_CACHE_BACKEND=memory python -m pytest -q

docker compose build
docker compose run --rm auth python -m app.scripts.seed
docker compose up -d --wait
```

В .env задаётся INTERNAL_JWT_SECRET; examples/imports подключается как /imports. Внешний GraphQL доступен на порту 8000. Внутренние gRPC-порты не публикуются; повторяемое чтение данных между сервисами выполняется с timeout и ограниченными retry, а изменяющие команды автоматически не повторяются.

Заключение

Межсервисный сценарий включает аутентифицированный gRPC, полноценные реакции Kafka consumers и реальную Reporting read model в пути GraphQL. Успешные события соответствуют сохранённым фактам, Analytics использует версию из события, а подтверждение offset не допускает потери записи при отказе DLQ.